Compute Flow Attribution over Dynamic Edge Microservices

Project-II (CS57004) report submitted to

Indian Institute of Technology Kharagpur

in partial fulfilment for the award of the degree of

Dual Degree (Btech+Mtech)

in

Computer Science and Engineering

by

Kalikiri Hrushikesh Reddy

(21CS30028)

Under the supervision of

Professor Sandip Chakraborty

Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur

Spring Semester, 2025-26

April 28, 2026

DECLARATION

I certify that

(a) The work contained in this report has been done by me under the guidance of my supervisor.

(b) The work has not been submitted to any other Institute for any degree or diploma.

(c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

(d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: April 27, 2026 (Kalikiri Hrushikesh Reddy)

Place: Kharagpur (21CS30028)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

KHARAGPUR - 721302, INDIA

CERTIFICATE

This is to certify that the project report entitled "**Compute Flow Attribution over Dynamic Edge Microservices**" submitted by **Kalikiri Hrushikesh Reddy** (Roll No. 21CS30028) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Dual Degree (Btech+Mtech) in Computer Science and Engineering is a record of bona fide work carried out by him under my supervision and guidance during Spring Semester, 2025-26.

Date: April 27, 2026

Place: Kharagpur

Professor Sandip Chakraborty
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Kharagpur - 721302, India

Abstract

Name of the student: **Kalikiri Hrushikesh Reddy** Roll No: **21CS30028**

Degree for which submitted: **Dual Degree (Btech+Mtech)**

Department: **Department of Computer Science and Engineering**

Thesis title: **Compute Flow Attribution over Dynamic Edge Microservices**

Thesis supervisor: **Professor Sandip Chakraborty**

Month and year of thesis submission: **April 28, 2026**

Modern distributed and telecom scale computing environments execute application requests across user space, kernel subsystem, and network infrastructure, forming complex execution paths that span compute and service flows. However, achieving end-to-end visibility across these layers remains challenging. Specifically, existing observability solutions operate in isolation: network monitoring, application tracing or kernel telemetry, providing only partial insights. Thus, operators face a fundamental limitation, either rely on coarse-grained statistics that miss fine grained behavior or deploy fragmented tools that fail to deliver unified cross layer correlation and per-user attribution.

In this project, we present a robust distributed observability framework that achieves fine-grained, per-request compute flow attribution . We look at ways for capturing, correlating, and analyzing compute flows for incoming server traffic across kernel space, user space, and network domains in all distributed endpoints to overcome existing limitations in flow visibility and granularity.

Acknowledgements

I would like to express my sincere gratitude to my MTP guide, Professor Sandip Chakraborty, for his constant support, insightful guidance, and invaluable feedback throughout the course of this project. Additionally, I extend my heartfelt thanks to the Department of Computer Science and Engineering, IIT Kharagpur, for providing me with this opportunity and the resources to pursue this research.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Cloud-native telecom platforms are increasingly used to host latency-sensitive services for 5G and 6G networks on large-scale microservice architectures. In this model, a single subscriber request does not execute within a single process or on a single machine, rather it traverses a chain of user-space service logic, kernel subsystems, and network infrastructure distributed across many physical hosts. Ensuring performance and meeting Service Level Objectives (SLOs) in such environments requires visibility across these cross-layer execution paths. Without a clear view of how a request propagates through the system, operators cannot determine whether a latency violation originates in application logic, kernel scheduling, data store access, or the network fabric between services.

In practice, however, achieving such visibility remains fundamentally challenging. Observability is inherently fragmented across domains: application-level tracing captures high-level request interactions at the service boundary, network monitoring provides flow-level statistics at the packet level, and kernel telemetry exposes low-level system events such as scheduling and memory allocation. These approaches operate in isolation, each providing only a partial and disconnected view of system behaviour. Operators are consequently forced into one of two unsatisfactory positions: they either rely on coarse-grained abstractions that aggregate activity at the

process level and miss fine-grained execution semantics, or they combine multiple disjoint tools whose outputs cannot be causally correlated, making accurate root cause analysis in distributed environments impractical.

This fragmentation is further compounded in telecom systems by the need for subscriber-aware observability. Subscriber-specific policies and Quality of Service requirements are maintained in backend repositories such as the Unified Data Repository (UDR). Existing observability frameworks do not integrate such subscriber context with runtime telemetry, preventing accurate per-user attribution of resource consumption, QoS enforcement, and SLO violation diagnosis. Without associating execution flows with specific user identities or sessions, operators cannot determine which subscriber's traffic is responsible for a resource spike or which user's SLA is being violated.

Recent advances in the extended Berkeley Packet Filter (eBPF) offer a promising foundation for addressing these challenges. eBPF enables programmable, fine-grained instrumentation by attaching lightweight, verifiably safe programs to kernel hook points, providing visibility across user space, kernel subsystems, and the network stack without requiring any modification to application code. However, existing eBPF-based approaches continue to operate in isolation. They capture detailed kernel telemetry but do not construct a unified execution view that correlates compute flows i.e CPU bursts, memory allocations, I/O operations with service flows i.e the RPC calls that carry a request across service boundaries. This leaves the fundamental gap in end-to-end causal visibility unaddressed.

Building a unified, user-aware observability framework for distributed systems surfaces four key system challenges. The first is **fine-grained attribution beyond process-level visibility**: in a distributed system, multiple applications share the same host and kernel, and existing monitoring systems capture activity at the process level, obscuring per-request execution context. The second is **cross-layer correlation across fragmented telemetry**: user-space, kernel, and network observations

must be fused into a causally consistent, continuous execution trace that spans all layers simultaneously. The third is **end-to-end tracing of compute and service flows**: application execution consists of both compute flows, which describe how a thread consumes resources on a single host, and service flows, which describe how a request propagates across hosts via inter-service calls. Correlating these two types of flows to reconstruct the full lifecycle of a distributed request requires mechanisms that go beyond what any single layer of the stack can observe alone. The fourth challenge is **subscriber-aware attribution**: associating execution flows with specific user or subscriber identities so that per-user resource consumption, QoS compliance, and SLA violations can be diagnosed with the necessary granularity.

In this project, we present a distributed observability framework that addresses these challenges through eBPF-based kernel instrumentation deployed at each node of a multi-machine microservice deployment. The framework captures fine-grained runtime events like scheduler decisions, memory allocations, network sends and receives, and inter-service RPC calls and correlates them into per-request execution records without requiring any modification to the monitored services.

The framework is evaluated on a twenty-five node deployment of DeathStarBench's Social Networking application, a widely used distributed benchmark whose inter-service communication patterns and data access behaviours closely resemble those of production microservice deployments. The evaluation demonstrates that the system successfully reconstructs causal call graphs spanning hundreds of spans across twenty-five machines, separates active CPU time from wall-clock latency at per-request granularity, and identifies dominant latency contributors including data store access patterns that are not visible through any conventional monitoring approach.

Chapter 2

Related Work

The problem of compute flow attribution and fine-grained observability has been addressed from various perspectives in prior literature and patents, though each approach exhibits significant limitations when applied to modern, multi-tenant, encrypted compute environments. This section reviews representative prior art and highlights the gaps that motivate the proposed solution.

2.1 Socket Transferring for HPC Networks

Choochotkaew and Chiba propose a method for transferring live socket connections across HPC hosts using kernel tracing to enable migration and fault tolerance Choochotkaew and Chiba (2024). While demonstrating kernel-level visibility for network operations, the approach focuses on socket state preservation rather than attribution. It does not correlate compute flows—spanning syscalls, CPU, disk, and network—to user intents or sessions, nor does it integrate with telecom control planes for end-to-end observability, SLA enforcement, or per-user charging Choochotkaew and Chiba (2024).

2.2 Mobile Device Traffic Attribution

Raleigh et al. present a system for identifying which mobile application generates network flows using socket metadata and process IDs Raleigh et al. (2017). However, the solution is limited to the network domain and does not extend to broader compute resources such as syscalls, CPU cycles, or memory consumption. It lacks integration with telecom subscriber data and does not support dynamic, intent-aware attribution for end-to-end compute activities crucial in multi-tenant edge environments Raleigh et al. (2017).

2.3 Flow Context-Based Traffic Classification

Zeng et al. propose statistical techniques for identifying encrypted proxy traffic using flow behavior analysis and machine learning Zeng et al. (2019). While effective for traffic classification, this work does not address attribution of compute resources to user sessions or intents. The reliance on heuristic methods restricts its ability to provide real-time, fine-grained resource usage mapping, and it does not support SLA enforcement or charging for in-network compute workloads Zeng et al. (2019).

2.4 Cilium: Container Network Observability

Cilium leverages eBPF to provide network observability and policy enforcement in Kubernetes environments Isovalent (2024). However, its attribution capabilities are limited to network flows and Kubernetes metadata, with no visibility into compute-level activities such as syscalls or CPU usage. The platform does not correlate these with telecom subscriber data or support fine-grained, per-user accountability required for 5G/6G and in-network compute deployments Isovalent (2024).

2.5 Summary of Limitations

Collectively, these prior works focus on isolated layers—network traffic, process-level monitoring, or container orchestration—without achieving comprehensive integration across compute, storage, and network resources. None provide real-time, per-user attribution spanning kernel events, user intents, and telecom subscriber contexts, which are essential for modern edge and in-network compute environments requiring fine-grained observability, SLA compliance, and usage-based billing.

Chapter 3

Tools and Frameworks used

To design and implement the observability system we used a set of frameworks and tools. This includes eBPF(extended Berkeley Packet Filter), BCC Framework in Python which provides eBPF capabilties and perf buffers to read/write data to eBPF maps from Python Processes, SSL encryption/decryption libraries.

3.1 eBPF (Extended Berkeley Packet Filter)

eBPF (Extended Berkeley Packet Filter) is a Linux kernel technology that allows small, sandboxed, event-driven programs to run safely inside the kernel. It provides a secure and efficient way to extend kernel functionality at runtime without modifying kernel source code or loading potentially risky kernel modules. Through this mechanism, developers can add features for observability, performance monitoring, security, and networking in a controlled and reliable manner.

Before eBPF, extending kernel functionality required recompiling the kernel or using kernel modules, both of which could compromise system stability. A single bug in a kernel module could crash the entire system. eBPF overcomes this limitation by

executing programs in a sandboxed environment that isolates them from the core kernel, ensuring that they cannot cause crashes or corrupt data.

Originally derived from the Berkeley Packet Filter (BPF) developed for packet filtering tools like `tcpdump`, eBPF has evolved into a general-purpose, in-kernel virtual machine. It includes a simple instruction set, a set of registers, and access to kernel data structures. This makes it useful not only for networking but also for tracing, monitoring, and security applications.

The safety and efficiency of eBPF are ensured by two main components: a **verifier**, which checks eBPF programs for safety before execution, and a **Just-In-Time (JIT) compiler**, which compiles the verified programs for fast execution within the kernel.

3.1.1 The eBPF Verifier

When an eBPF program is loaded into the kernel, it first goes through a static analysis process known as the **verifier**.It ensures that every program is safe to execute before it runs. It performs several checks to verify that the program cannot cause instability or security issues in the kernel. The main checks performed by the verifier include:

- **Guaranteed Termination:** The verifier analyzes the program's control flow to ensure there are no infinite loops. This guarantees that the program will always finish execution and cannot hang or block the kernel.
- **Memory Safety:** The verifier checks every memory access to confirm that it is within valid bounds, such as the program's stack or permitted eBPF maps. This prevents unauthorized access to kernel memory.
- **State Validation:** It tracks the state of all registers and stack variables to ensure that no uninitialized variables are read or used during execution.

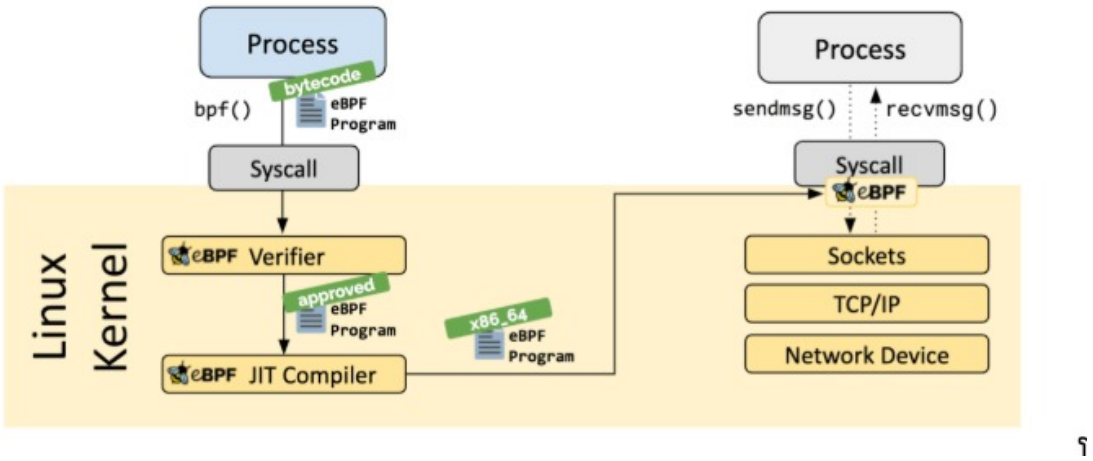

FIGURE 3.1: eBPF Verifier Architecture [eBPF.io (2024)]

- **Restricted Function Calls:** eBPF programs are only allowed to call a predefined set of safe helper functions provided by the kernel (for example, functions to read the current time or look up map elements). The verifier ensures that only these approved functions are called.

An eBPF program is loaded into the kernel only if it passes all these checks. If any rule is violated, the verifier rejects the program, ensuring that only safe and well-behaved code is executed inside the kernel.

3.1.2 The Just-In-Time (JIT) Compiler

After an eBPF program successfully passes the verification stage, it is processed by the **Just-In-Time (JIT) compiler**. The JIT compiler translates the eBPF bytecode into native machine code specific to the host system's architecture (for example, x86 or ARM). This translation enables the eBPF program to run directly on the hardware, achieving near-native execution speed.

3.1.3 Hook Points

eBPF programs are event-driven and do not run continuously. They remain inactive in the kernel until attached to specific **hook points** in the kernel or user-space execution path. When the corresponding event occurs, the kernel automatically triggers the attached eBPF program. Some of the most frequently used hook points include:

- **kprobes and kretprobes:** These hooks attach to the entry (`kprobe`) or return (`kretprobe`) of almost any kernel function, making them useful for kernel-level debugging and performance tracing.
- **uprobes and uretprobes:** Similar to kprobes, these allow eBPF programs to attach to the entry (`uprobe`) or return (`uretprobe`) of user-space functions. They enable observability in user applications without requiring modifications to the program's source code.
- **Tracepoints:** Fixed, low-overhead hook points placed at important locations in the kernel code, such as process scheduling or context switching events.
- **XDP (eXpress Data Path):** A networking hook that runs in the network driver layer before packets reach the network stack, supporting fast packet filtering and processing.

3.1.4 Data Sharing: eBPF Maps

For eBPF programs to be effective, they must be able to store state and exchange data with other parts of the system. This is achieved using special data structures called **eBPF maps**. Maps are the primary mechanism for data sharing between the kernel and user space. They allow eBPF programs to maintain state across multiple executions, share data between different eBPF programs, and communicate efficiently with user-space applications.

eBPF maps act as flexible key-value stores managed by the kernel. Several types of maps are available, including arrays, hash maps (`BPF_MAP_TYPE_HASH`), per-CPU arrays, stack traces, queues, and ring buffers (`BPF_MAP_TYPE_RINGBUF`) that enable high-speed, lock-free data streaming. This flexibility supports bidirectional data exchange between kernel and user space:

- **Kernel to User Space:** eBPF programs can write information such as metrics or events into a map, which user-space applications can then read. For high-frequency data, a ring buffer map is often used to transfer data efficiently without locking.
- **User Space to Kernel:** User-space processes can update map entries with configuration data or control values (for example, threshold values or lists of blocked IP addresses). The eBPF program can then read this data to modify its behavior dynamically without reloading.

3.2 BCC Framework for eBPF in Python

The BPF Compiler Collection (BCC) is a comprehensive toolkit for creating efficient kernel tracing and manipulation programs using extended Berkeley Packet Filter (eBPF) technology. BCC provides a Python interface that simplifies the development of eBPF programs while maintaining the performance benefits of in-kernel execution. The framework serves as the foundational infrastructure for all kernel-space instrumentation in this work.

3.2.1 Architecture and Capabilities

BCC abstracts the complexity of eBPF program compilation, loading, and verification through a high-level Python API. The framework translates Python-embedded

C code into eBPF bytecode, which is then compiled, verified by the kernel's eBPF verifier, and dynamically loaded into the kernel. This architecture enables rapid prototyping and iteration while ensuring safety guarantees enforced by the kernel's verification process. BCC provides built-in support for Tracepoints , Kprobes , Uprobes, BPF Maps and Perf Ring Buffers which are Low-Latency event streaming mechansims for various eBPF programs to communicate with Python Coordination processes.

The Python control program interfaces with BCC through the `BPF` class, which handles eBPF program compilation, map manipulation, and event polling. This architecture separates complex parsing and aggregation logic (maintained in user space) from performance-critical attribution logic (executed in kernel space), optimizing both correctness and efficiency.

3.3 SSL Modules and Library Architecture

The system achieves transparent HTTPS traffic attribution by instrumenting the OpenSSL library, which serves as the de facto standard for TLS/SSL encryption across numerous programming languages and applications. This section details the SSL library architecture and the rationale for targeting OpenSSL specifically.

3.3.1 OpenSSL Library Structure

OpenSSL 3.x is dynamically linked by Python's `ssl` module, Node.js's `crypto` module, and system utilities such as `curl` and `wget`. The library resides at `/usr/lib/libssl.so.3` on the test system (Arch Linux with kernel 6.1.x) and exposes standardized C API functions for SSL/TLS operations. The system targets two primary read functions:

- `int SSL_read(SSL *ssl, void *buf, int num)`: Traditional read interface returning the number of bytes read

- `int SSL_read_ex(SSL *ssl, void *buf, size_t num, size_t *readbytes)`: Extended interface providing success/failure status with byte count via pointer

These functions represent the decryption boundary where plaintext HTTP headers become accessible after TLS processing. By instrumenting these functions with uprobes, the system captures decrypted data transparently without requiring application modification or compromising encryption.

3.3.2 Compatibility and Generality

The implementation specifically targets OpenSSL 3.x but is architecturally compatible with any OpenSSL version exposing `SSL_read` and `SSL_read_ex` symbols. Table 3.1 summarizes the compatibility matrix for various applications and SSL libraries.

Table 3.1: SSL Library Compatibility Matrix

Application	**Language**	**SSL Library**	**Compatible**
Python HTTPS server	Python	OpenSSL via ssl module	✓
Node.js HTTPS server	JavaScript	OpenSSL via crypto module	✓
Nginx web server	C	OpenSSL	✓
Apache with mod_ssl	C	OpenSSL	✓
Go HTTPS server	Go	Go's crypto/tls (uses OpenSSL)	✓*
curl	C	OpenSSL	✓
wget	C	OpenSSL	✓
Firefox/Chrome	C++	NSS (Network Security Services)	×
Java applications	Java	JSSE (Java Secure Socket Extension)	×
Go with BoringSSL	Go	BoringSSL	×
Rust with rustls	Rust	rustls (Pure Rust TLS)	×
mbedTLS applications	C	mbedTLS	×

* Go's default configuration on Linux uses the system's OpenSSL library.

3.3.3 Extension to Alternative TLS Libraries

While the current implementation targets OpenSSL, the uprobe-based interception model generalizes to alternative TLS libraries by adapting the target function signatures:

- **NSS (Firefox/Chrome)**: Target `PR_Read` and `PR_Write` functions
- **JSSE (Java)**: Requires JVM-internal instrumentation using agents or instrumentation frameworks
- **BoringSSL**: Similar API to OpenSSL with minor signature differences
- **mbedTLS**: Target `mbedtls_ssl_read` function

The modular design isolates SSL interception logic, enabling straightforward adaptation to diverse TLS implementations without modifying the core attribution architecture.

3.4 DeathStarBench and the Social Network Application

DeathStarBench is an open-source benchmark suite developed by Cornell University for evaluating microservice infrastructure and observability tooling under realistic cloud-native workloads. It provides several end-to-end applications including a social network, a media service, and a hotel reservation system each built from multiple independently deployed services that communicate over well-defined RPC interfaces. The benchmark is widely used in systems research because its service interaction patterns, fan-out structures, and data access behaviors closely resemble those found in production internet-scale deployments.

The Social Network application within DeathStarBench was selected as the evaluation target for this work. It consists of twenty-five backend services including post composition, timeline management, user graph traversal, media handling, and unique ID generation that interact through Apache Thrift RPC over plain TCP on port 9090. A single user action such as composing a post triggers a fan-out of concurrent Thrift calls to up to nine of these services simultaneously, with several

services making further calls to Redis, MongoDB, and Memcached data stores. This multi-tier, multi-service structure makes it a demanding testbed for distributed attribution: correctly reconstructing the causal call graph requires matching dozens of Thrift spans across up to fifteen physical machines within a single request's lifetime.

The choice of Thrift as the inter-service protocol was directly relevant to the scope of the attribution framework implemented in this project. All inter-service communication within the Social Network application uses the Thrift binary framing that the eBPF interception subsystem is designed to decode, ensuring that every RPC boundary in the application is observable without any modification to the service binaries. The workload generator included in the benchmark which issues configurable numbers of concurrent HTTP POST requests to the Nginx ingress was used directly to drive the evaluation described in Chapter 5.

Chapter 4

Design and Implementation Details

We designed the Compute Flow Attribution framework as a highly distributed system comprised of dynamic modules strategically attached to both native kernel hook points and user space library probes. To bridge the massive semantic gap between low level hardware execution constraints and high-level application intents, we implemented a dedicated, privileged user-space process referred to securely as the User-Space Coordinator (USC). The USC acts as the central cognitive engine on each host node, explicitly designed to ingest thousands of asynchronous eBPF peripheral events and bring them all together into a synchronized, highly detailed transactional outlook.

Crucially, the entire implementation operates on a strict "black box" methodology, requiring absolutely zero modifications to the underlying microservice source codes. As application requests arrive at the host, the framework elegantly intercepts them at the earliest possible socket layer, extracting cryptographic headers via *uprobes* and network 4-tuples via *tracepoints* to perfectly identify the transaction. The USC natively pins this Request ID to the executing Thread Identifier (TID), ensuring all subsequent physical memory allocations, storage I/O, and CPU silicon cycles are

irrefutably tracked. By deploying these programmatic eBPF agents simultaneously across multiple nodes, the local USC instances can continuously stream structured, thread-pinned telemetry to a central aggregation handler to seamlessly reconstruct the overarching distributed call graph.

4.1 Architecture Outline

4.1.1 Introduction and System Vision

Modern microservice architectures introduce severe operational complexity, primarily due to the deeply nested and ephemeral nature of distributed requests. Traditional profiling techniques either rely on heavy instrumentation which alters application source code and introduces significant overhead or network sniffing, which loses visibility into intra-process resource consumption and encrypted traffic.

The developed profiling system presents a highly transparent, low-overhead distributed tracing and resource tracking architecture. By leveraging Extended Berkeley Packet Filter (eBPF) technology, the system hooks deeply into the Linux kernel to intercept system calls, memory allocations, and network-level events without requiring any source code modifications to the deployed microservices. The core philosophy of this architecture is **Request-Centric Resource Attribution**, which securely attributes hardware utilization (CPU bursts, memory footprints, and disk I/O) directly to individual, distributed HTTP/Thrift requests as they traverse a multi-node cluster.

4.1.2 End-to-End High Level Architecture

To maintain low latency and prevent localized bottlenecks, the system follows a federated aggregation model. The architecture is logically segmented into three

major tiers:

- **The Kernel-Space eBPF Subsystem:** Deep probes acting as invisible sensors attached to system calls and logical execution tracepoints.
- **The User-Space Coordinator (USC) Daemon:** A per-node agent process responsible for contextualizing raw kernel data and dispatching it.
- **The Central Log Handler:** A global observation node tasked with ingesting node-local metrics and synthetically stitching them together into a meaningful Distributed Acyclic Graph (DAG) of the various backend service calls made.

4.1.3 Subsystem Interactions and Integration

4.1.3.1 The eBPF Subsystem (Kernel-Space Sensors)

Rather than a monolithic kernel module, the eBPF layer is disjointed into specialized components. User-space probes (Uprobes) handle encrypted payload interception (e.g., `SSL_read`/`SSL_write`), unlocking the ability to inspect HTTP layer headers securely before they execute within the application layer. Parallel tracepoints handle asynchronous hardware interactions tracking CPU context switches, packet transmissions, and process lifecycle modifications.

Data inside the kernel is accumulated natively via **eBPF Maps**, mitigating the overhead of transferring every atomic event to user-space. Only essential chronological boundaries such as connection creations, thread exits, and header blocks are emitted via **Perf Ring Buffers**.

4.1.3.2 The User-Space Coordinator Daemon (USC)

The USC daemon acts as a logical bridge between kernel-level events and application-level context. Its primary functions include:

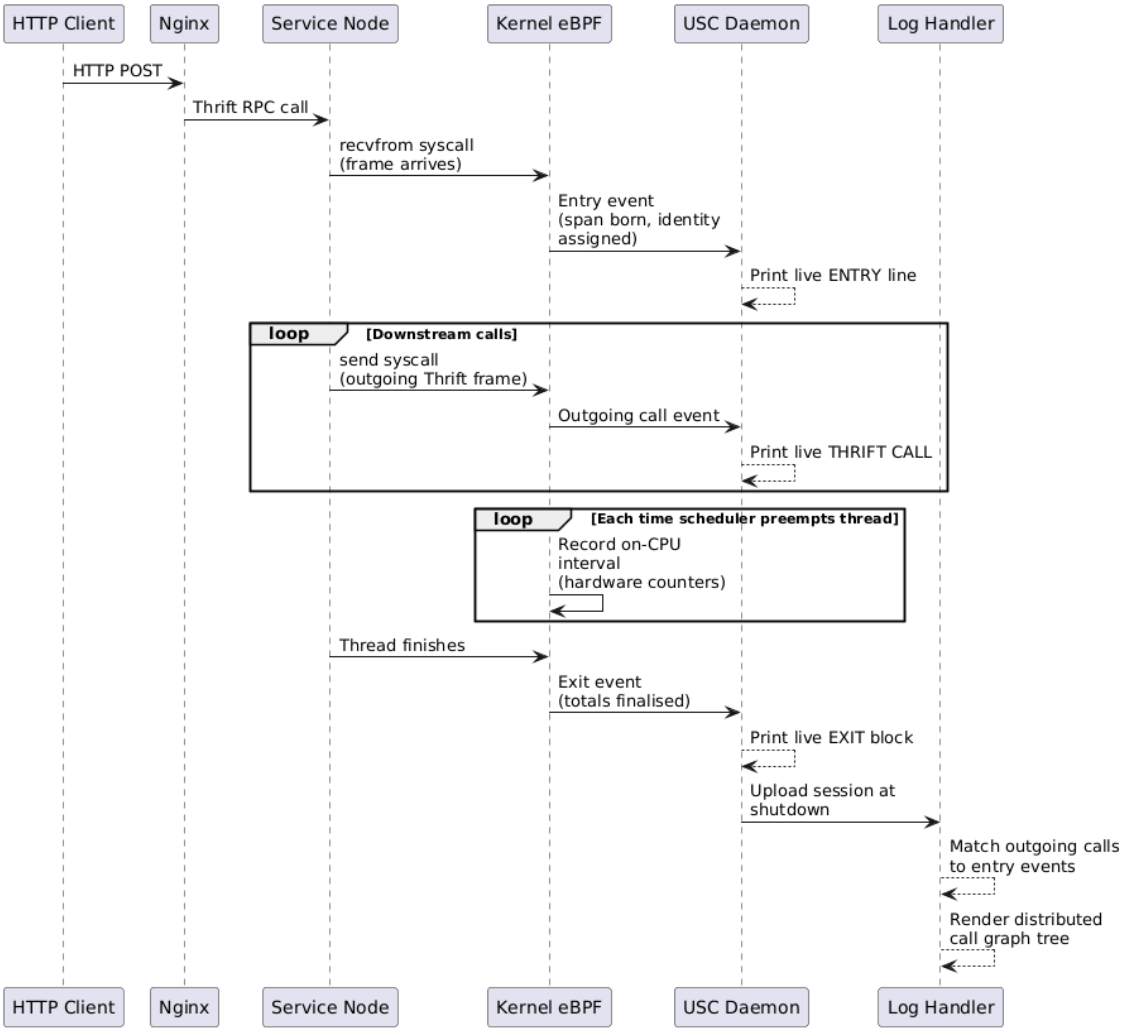

FIGURE 4.1: Complete high level Architecture

- **Header Parsing and Thread Pinning:** When the kernel intercepts raw TCP packets or decrypted SSL buffers, the USC parses the application-layer headers to extract critical propagation identifiers, primarily the X-Request-Id and incoming network 4-tuples. Once identified, the daemon synchronizes this request context dynamically with the exact Operating System Thread ID (TID) currently executing the request via shared eBPF memory maps, bridging the semantic gap between application and kernel.

- **Resource Contextualization:** As the tracked application thread executes, it natively incurs resource consumption spanning exact processor cycles, mapped

virtual memory expansions, and block storage I/O operations. Through the shared eBPF maps initialized by the USC, the kernel natively buckets these hardware tolls against the specific X-Request-Id pinned to that thread, ensuring all physical utilization is mathematically tethered back to the logical transaction.

- **Outbound Detection:** In highly distributed microservice architectures, an incoming request often recursively triggers downstream outbound RPC requests. The USC actively monitors outgoing connections and intelligently dissects serialized binaries (such as strict Thrift RPC frames) to detect outbound backend calls. Extracting sequence IDs and method names directly off the wire allows the USC to log subsequent service-to-service propagations seamlessly.

- **Lifecycle and Hand-off:** To maintain bounded memory usage, the system correctly models request lifecycles. When a pinned application thread finishes a request block whether by terminating cleanly or being recycled back into an async thread-pool, the USC detects the conclusion. It forcefully harvests all accumulated kernel metrics (such as precise CPU burst counts, peak physical memory consumption, network byte transmissions, and overall response latencies) into unified node-local JSON payloads, purging the eBPF maps for subsequent request cycles.

4.1.3.3 Central Log Handler and DAG Synthesis

While each node's USC maintains a localized lens of resource usage, a microservice transaction spans multiple networked containers. To resolve this, the local USC processes continuously dispatch the collected logs at that site to the **Central Log Handler**.

The central log handler implements a deterministic Distributed Acyclic Graph (DAG) construction algorithm. When Service A calls Service B, the system records an outbound connection event at the source and an inbound request event at the destination. These events are correlated via robust **TCP 4-tuple matching** at the central log handler which receives all the distributed logs. This architecture yields an interactive tree structure plotting the microscopic end-to-end traversal of a request, mapped to precise end-to-end cumulative resource usage.

4.2 The Connection Attribution Subsystem

The Connection Attribution subsystem acts as the fundamental networking anchor within the observability framework. While the system reliably tracks the foundational correlation of incoming user traffic to active thread contexts, its most critical mandate is projecting this visibility outward across distributed network boundaries. Modern microservices rarely execute entirely in isolation; a single ingress request frequently cascades into entirely new, outbound Remote Procedure Calls (RPCs) to distinct backend services or database layers.

The primary focus of this subsystem's architectural evolution is seamlessly bridging these outgoing "Single Links" (Service A $\rightarrow$ Service B). Achieving this natively without breaking encryption, modifying application logic, or adding heavy service mesh proxies is notoriously difficult. By dynamically tracking kernel-level file descriptor states and deep-diving into outbound binary protocol structures perfectly in-flight, the framework successfully traces deeply nested distributed interactions. Below, we briefly recap the baseline ingress connection 4-tuple attribution logic, before comprehensively exploring the extended architecture required to flawlessly intercept, dissect, and attribute outbound Apache Thrift payloads and database executions.

4.2.1 Ingress Packet Parsing and 4-Tuple Attribution

The original iteration of the connection attribution subsystem was architected to securely associate every unit of incoming decrypted (or plaintext) traffic with its exact source and destination TCP 4-tuple. Initially, early prototypes attempting to cache properties during socket `accept()` calls suffered from pervasive race conditions. In a high-throughput server where hundreds of connections are accepted across thread pools simultaneously, asynchronous lookup operations caused mismatch misattributions up to 5% of the time which is a platform-breaking margin for billing and forensic telemetry.

To eliminate temporal race conditions, the existing logic transitioned to *Dynamic Deep Kernel Traversal*:

- **Tracepoint Hooking:** The eBPF agents are attached exclusively to raw socket ingress operations (`sys_enter_recvfrom` and `sys_enter_read`).
- **Real-time Kernel Traversal:** At the exact instance a thread triggers a read against a file descriptor, kernel constructs have definitively mapped that FD to the live network endpoint. The logic verifies the family is `AF_INET` and then atomically traverses the active object references: `task_struct` → `files_struct` → `socket` → `inet_sock`.
- **4-Tuple Construction:** Upon reaching the `inet_sock`, the framework decodes the definitive `inet_saddr`, `inet_daddr`, `inet_sport`, and `inet_dport` safely, using `bpf_ntohs` to handle byte-order conversions correctly for userspace consumption.
- **State Maintenance:** The tuple is stored synchronously in two core eBPF Hash Maps: `fd_to_conn` (keyed by the compound Process-FD integer pid_fd) and `tid_to_fd` (keyed by Thread ID pid_tgid).

This ensures a race-free foundation. There are no timing windows nor context switches that could misappropriate a client's ingress packet before it becomes successfully attached to the processing Thread ID.

4.2.2 Outbound RPC Interception (Link Service $A \rightarrow B$)

While the existing logic addressed inbound user-facing traffic, microservice infrastructures primarily rely on complex, cascaded external backend service calls. A single inbound request to Service A frequently triggers secondary internal Remote Procedure Calls (RPCs) to Service B or database layer endpoints. The most critical addition to the framework is the capability to seamlessly track and identify these outgoing single links without intercepting or slowing outbound network bandwidth. To accomplish this, the framework was extended to identify outgoing Apache Thrift binary protocols and database calls.

4.2.2.1 Pervasive Egress Tracepoint Insertion

To detect outgoing payloads, the subsystem broadened the array of tracepoints monitored to include the entirety of the Linux socket egress path: `sys_enter_sendto`, `sys_enter_write`, `sys_enter_writev`, `sys_enter_sendmsg`, and `sys_enter_sendmmsg`. Whenever a worker thread associated with an active inbound request attempts to push data via these syscalls, the extended eBPF hooks instantaneously evaluate the targeted socket structure and intercept the `AF_INET` buffer before it descends the network stack.

4.2.2.2 Transparent Thrift Binary Dissection

The focal point of the extended logic is the native invocation of a custom C helper executing within the kernel tracepoints. Unlike standard HTTP which leverages basic string parsing, Thrift uses condensed encoding frameworks.

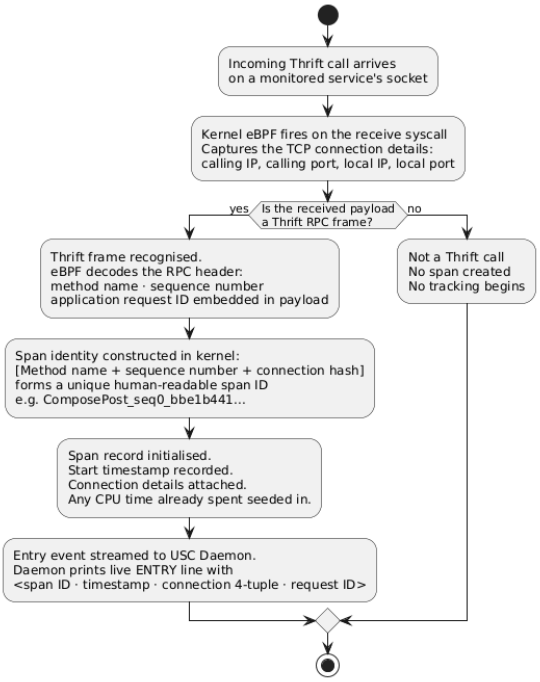

FIGURE 4.2: Processing Incoming Network Traffic at registered Nodes.

1. **Protocol Signature Detection:** The kernel reads the leading 8 bytes of the egress memory buffer. It interrogates both *Strict Framed Binary* ($hdr[4..6] == 0x800100$) and *Strict Unframed Binary* ($hdr[0..2] == 0x800100$) signatures, accounting for both header-shifted and non-header environments.

2. **Metadata Extraction:** Because microservice-to-microservice internal RPC calls frequently traverse private virtual networks unencrypted, the kernel leverages deep struct offset analysis. It dynamically applies `bpf_ntohl` to decode the `name_len` to capture the precise RPC method string. It calculates the `seq_id_offset` scaling the buffer offset over the variable name byte-length to extract the 32-bit internal sequence block reliably.

3. **Deep Payload Inspection:** The framework probes within the binary layout for Field ID #1 cast as a 64-bit integer type ($0x0A$). If located, it extracts a bespoke `payload_req_id` variable, guaranteeing cross-node identity matching even when IP/Port TCP streams fail identification logic.

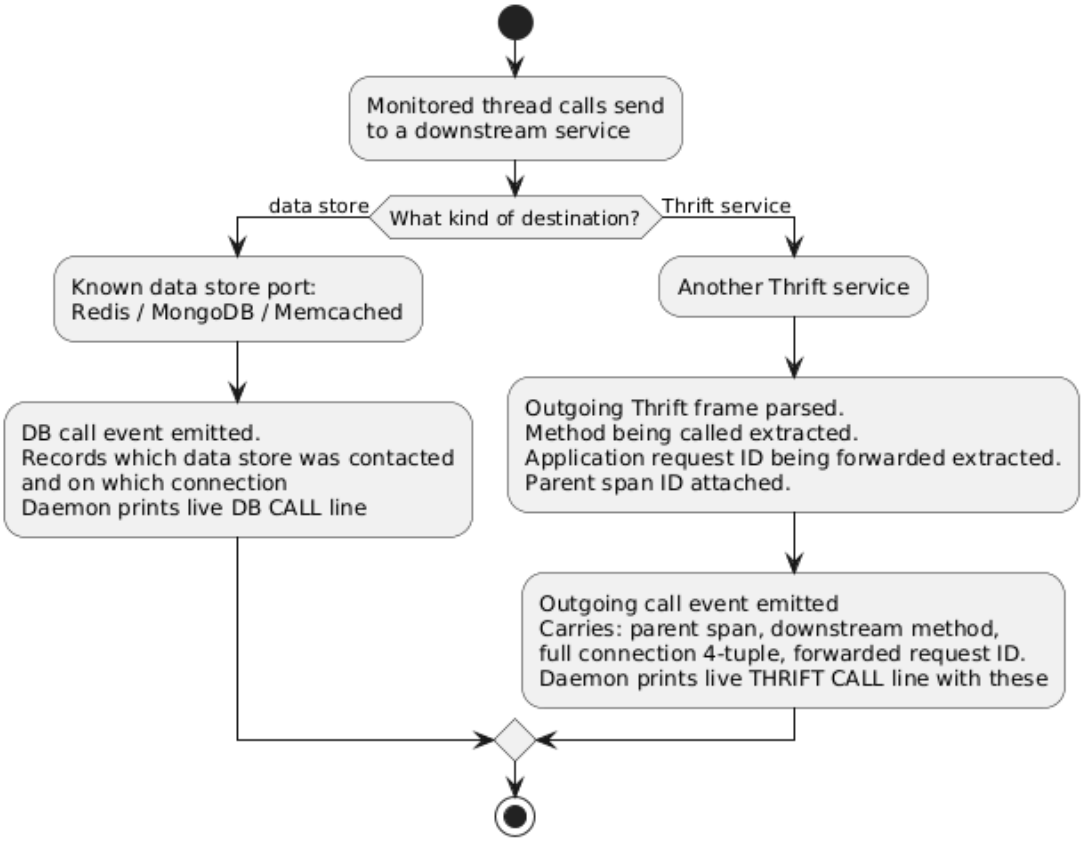

FIGURE 4.3: Processing Outgoing Network Traffic at registered Nodes.

4.2.2.3 Edge Stitching: Bridging the Distributed DAG

Extracting the outbound RPC method is insufficient without attributing it to the originating caller. As the detection proceeds, it queries the `tid_to_request_id` eBPF map. Because the existing ingress logic pinned the inbound parent Request ID to the active Thread Identifier, the outbound kernel extension extracts this `parent_request_id` and overlays it directly onto the newly tracked outbound Thrift Call.

4.2.2.4 External Entity Attribution (Databases)

The framework captures database communication by inspecting destination ports statically (e.g., 27017 for MongoDB, 6379 for Redis, and 3306 for MySQL). Instead of decoding diverse wire protocols which would increase BPF verification complexity, it tags these outgoing links as a `DB_OP` with a pseudo sequence identifier.

4.2.2.5 Final Linking

To maintain extreme latency tolerances, the outgoing connection records are packaged into a C-struct (`thrift_out_event_t`). This unified struct aggregates the executing *tid*, extracted *parent_request_id*, egress TCP 4-tuple, RPC method, tracked *seq_id*, and *payload_req_id*. This payload is transmitted via optimized eBPF Perf Ring Buffers to the userspace coordinator script.

The coordinator digests these metrics by labeling the traffic with unique child prefixes before dispatching them toward the central log handler process.The central log handler receives information about the backend call observed on both the caller and receiver side which it later stitches together to make the final link. As Service A broadcasts a Thrift event toward Service B, the handler correlates Service A's outbound parameters against Service B's inbound boundaries, automatically synthesizing microscopic Distributed Acyclic Graphs (DAGs) across process and host boundaries.

4.3 The Resource Tracking Subsystem

The Resource Tracking subsystem constitutes the analytical engine of the observability framework. In abstracted microservices and dense container environments, correlating hardware utilization directly to disparate network requests is deeply obfuscated by thread pooling, asynchronous multiplexing, and OS-level virtualization.

The Resource Tracking subsystem securely attributes computational taxation from CPU cycles to exact physical page mappings solely to individual requested flows natively in the Linux kernel.

By unifying system calls, hardware performance counters, and `kmem` allocators, the agent produces extreme, thread-pinned fidelity. Below we detail the precise methodologies and interpretational mechanics used to extract every metric observed during a distributed execution trace.

4.3.1 The Tracking Methodology

The overarching philosophy of the resource tracking subsystem is defined by **Bounded execution boundaries**. Tracking is neither continuous nor systemic it is exclusively activated the moment the Connection Attribution Subsystem formally registers an inbound Network Request Identifier onto a specific executing core Thread (`TID`). The eBPF tables initialize a blank `resource_usage_t` struct, and every subsequent operation invoked by that distinct `TID` natively contributes to the designated request bucket until the thread exits or completes processing.

4.3.2 Details of Tracked Metrics

To illustrate the subsystem's output and methodology comprehensively, we examine the following representative log entry generated upon a thread's completion:

Table 4.1: Detailed Resource Tracking Log Metrics

Metric Name	**Observed Value**
Request-ID	`ComposeText_seq0_010037f97c10e0ec`
Date & Time	2026-04-05 22:03:12
Service (PID)	`socialnetwork-text-service-1` (356693)
Thread ID (TID)	359514
Source IP:Port	172.18.0.19:53476
Destination IP:Port	172.18.0.21:9090
Virtual Memory	16524.0 KB
Peak Physical Memory	140.0 KB
Network Sent	0.3 KB
Network Received	0.1 KB
Disk Read	0.0 KB
Disk Write	0.0 KB
CPU Bursts	6
Avg Burst Duration	0.079 ms
Total CPU Time	0.473 ms
Total CPU Cycles	1,657,509
Total Instructions	1,190,333
Framework Overhead	13,447 ns
Response Latency	0.471 ms

4.3.2.1 Identifiers and Topographical Context

- **Request-ID** : The universally unique identifier tracking this distributed execution. It natively couples the Thrift RPC method (for example `ComposeText` in Table 4.1), the sequence identifier , and an extracted payload hex context. This acts as the primary key bridging kernel eBPF structs to global distributed DAG logs.
- **Date & Time** : The wall-clock completion timestamp.
- **Service**: Identifies the exact application process processing the request, combining the containerized service name and its kernel Process ID.

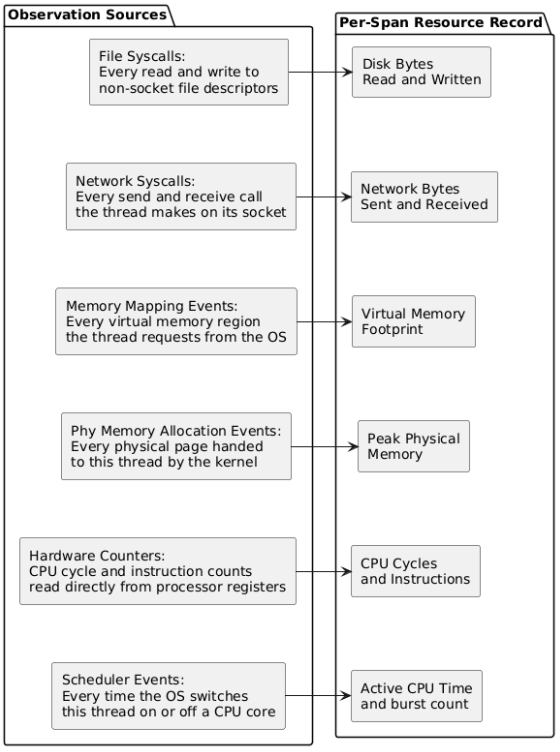

FIGURE 4.4: Tracking methodology overview.

- **Thread** : The exact OS-level Thread ID (`TID`) responsible for executing the workload.
- **Source/Dest IP:Port**: Extracted precisely at the `sys_enter_read` phase by deep struct traversal. Confirms the exact TCP 4-Tuple networking overlay (often indicating internal Docker container virtual networks).

4.3.2.2 Virtual and Physical Memory Signatures

- **Virtual Mem(KB)**: Operating systems vastly over-allocate memory virtually compared to what is physically backed by RAM. Tracked by intercepting `sys_enter_mmap` and `sys_enter_mprotect`, the framework strictly isolates requested pages holding `PROT_READ|PROT_WRITE` and `MAP_PRIVATE|MAP_ANONYMOUS` flags. This shows the application requested roughly ∼16MB of working space in Table 4.1.

- **Peak Phy(KB)**: While virtual memory is an abstraction, Peak Physical Memory denotes strict hardware DDR utilization. Relying on userspace memory pollers (such as reading `/proc/<pid>/smaps`) is far too slow and often misses transient spikes inside microsecond-length RPC requests.

  To extract this with flawless nanosecond fidelity, the architecture implements a **Deterministic Page-Level Tracking Strategy**:

  1. **Allocation Hook (`kmem:mm_page_alloc`)**: When the kernel grants physical memory, the agent hooks this tracepoint to extract the granted Page Frame Number (`PFN`) and page `order`. It computes the exact hardware bytes allocated via the standard buddy-allocator math: `[(1ULL << order) * 4096]`.
  2. **Page Ownership Mappings**: The framework updates an eBPF Hash Map (`pfn_owner`), securely binding the bare-metal `PFN` exclusively to the executing Thread ID (`TID`), while simultaneously mapping it to the Request-ID via `pfn_to_request_id`.
  3. **Live Watermark Calculation**: The thread's live physical footprint tracker (`tid_curr_phys`) is instantaneously incremented. The framework then checks the actively tracked `request_resources` struct. If the newly incremented physical footprint exceeds the previously registered `peak_physical_bytes`, the peak watermark is actively pushed.
  4. **Release Hook (`kmem:mm_page_free`)**: As the process returns memory chunks, the tracepoint dynamically looks up the target `PFN` from the `pfn_owner` map, identifies the exact original thread, and aggressively decrements the active footprint (`tid_curr_phys`).

  By synchronously calculating the deltas of absolute hardware page blocks natively upon assignment, the agent easily catches ultra-transient peaks. It

proves that despite requesting a bloated 16MB abstracted virtual map overhead, exactly 140.0 KB of fundamental DDR execution memory was concurrently committed by the motherboard to accomplish this Thrift method.

4.3.2.3 Explicit I/O Stream Mapping

Unlike traditional monitoring agents which aggregate overarching block-device or network-interface statistics via `/proc` polling or interface tracing, this architecture strictly isolates exact Application-Layer bounds. It traces native I/O directly at the exact interface between User-Space execution and Kernel-Space hardware dispatch.

- **Network I/O (Send & Recv)**: When a thread invokes an input/output payload via networking system calls (`sys_enter_recvfrom`, `read`, `sys_enter_sendto`, `write`, `sendmsg`, etc.), the eBPF framework dynamically intercepts the trace.

    1. **Socket Resolution**: The initial kernel tracepoint aggressively evaluates the targeted File Descriptor (`FD`). If the internal structures confirm a socket boundary (such as `AF_INET` for external IPv4 or `AF_UNIX` for localized meshes), the agent triggers an internal `active_sock_op` execution state indicating a network reception or transmission.
    2. **Byte Delta Accumulation**: The framework securely couples this entry phase with explicit `sys_exit` handler hooks. Upon successful completion, if the OS return code evaluates positive (`ret > 0`), the exact number of byte lengths successfully pushed or pulled through the OSI layers are instantaneously, atomically aggregated into the parent `request_resources` struct.

- **Block Storage I/O (DiskRd & DiskWr)**: Crucially, standard Linux POSIX calls (like `read` and `write`) are structurally overloaded to handle both remote sockets and localized disk paths.

1. **Object Type Isolation**: By dynamically interrogating the Kernel's `file_operations` object behind the File Descriptor during the `sys_enter` hook, if the struct verifies standard block storage operations rather than networking objects, a distinct disk-centric `active_sock_op` flag is raised.
2. **Throughput Separation**: Upon returning from the backing disk controller context inside `sys_exit`, the exact payload bytes retrieved from solid-state drives or flushed to log files are bucketed identically but entirely partitioned from the active network stream, securely accumulating exclusively into the `disk_read_bytes` and `disk_write_bytes` telemetry ledgers.

4.3.2.4 Extreme CPU Analytics (PMU Tracking)

Traditional process-level tick evaluations fail severely in distributed microservices because threads hyper-actively sleep or yield during asynchronous networking. To conquer this without synthetic estimations, the architecture intercepts the kernel's deepest schedule dispatcher (`sched:sched_switch`) to precisely profile direct hardware engagement.

Because probing both thread wake-ups and thread-sleeps natively is frequently inhibited by kernel constraints (e.g., `finish_task_switch` often being marked `notrace`), this framework evaluates both the outgoing (`prev`) and incoming (`next`) thread transitions symmetrically within a single atomic tracepoint invocation.

- **Hardware PMU Bounding**: When a monitored thread is scheduled *ON* to a processor core, the framework instantly queries the bare-metal **Performance Management Units (PMUs)** via `perf_cycles.perf_read()` and `perf_instructions.perf_read()`. It caches these absolute counter baselines natively inside a highly constrained `cpu_burst_start_map`.

- **Delta Computation (Total CPU, Cycles, Instructions)**: When the kernel scheduler forcibly evicts the thread *OFF* the processor, the incoming tracepoint invokes against the active core and compares the current hardware PMUs against the previously cached baselines. It mathematically extracts the absolute monotonic operational nanoseconds (`bpf_ktime_get_ns()`), the pure oscillator clock cycles consumed, and the strictly retired binary instructions executed at the CPU silicon layer. These strict deltas perfectly bound the physical pipeline efforts, rendering the tracking explicitly immune to idle times, spin locks, or I/O suspension interference.

- **Burst Counting & Averages**: The framework explicitly tracks how many microscopic scheduler shifts ("Bursts") successfully occurred while processing the isolated network request. By mathematically correlating average computation durations against raw bursts, the system allows operators to deduce deep thread health: erratic short durations or profoundly high burst counts intrinsically highlight contentions, lock-spinning, or aggressive asynchronous I/O dragging down native performance.

- **Pre-Assign State Guards**: To guarantee mathematically flawless aggregations across aggressive intra-core handoffs, the raw CPU PMU telemetry is first shielded into an intermediary `tid_cpu_pre_assign` map. It is subsequently flushed explicitly into the parent network-request resource bucket inside the kernel, precluding any user-space synchronization failures.

4.3.2.5 Latency and Architectural Overhead Isolation

A critical vulnerability of traditional observability sidecars is their inherent inability to distinguish between actual application execution latency and the artificial wait-time injected by the observability framework itself. This eBPF architecture explicitly isolates and exports both bounds directly from the kernel ledgers.

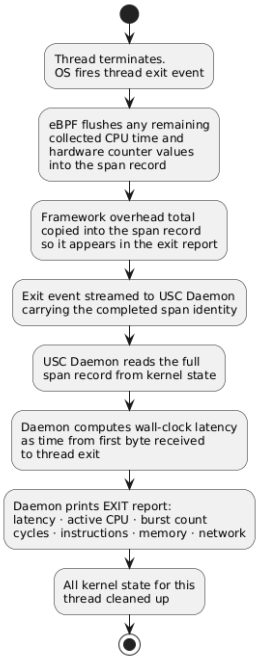

FIGURE 4.5: Thread exit and cleanup overview.

- **Response Latency Computation**: Rather than coarsely evaluating the thread lifecycle from spawn to termination (which fundamentally captures idle thread-pool waiting artifacts and TCP keepalives), the framework traces exact computational transaction bounds. Latency is analytically constrained as the absolute monotonic difference between the **First Network Read** (`first_recv_ns`), the exact microsecond the kernel unwraps the incoming HTTP headers or Thrift sequence arrays and the **Last Network Dispatch** (`last_send_ns`), the precise moment the OS clears the outgoing buffer back to the OSI datalink layer. This completely isolates active thread response efforts.

- **Granular Framework Overhead**: To guarantee absolute operational transparency, the architecture mathematically calculates its own "Observer Effect" natively. Every single isolated eBPF tracepoint invoked during the thread's execution runs a dedicated micro-timing sequence. Upon entry of the hook, an absolute initial timestamp (`bpf_ktime_get_ns()`) is captured. Moments later,

immediately preceding the return of control-flow back to the application context, a secondary timestamp is logged. The resulting computation is natively pushed into a persistent `tid_to_overhead_ns` eBPF accumulator map.

By exporting this cumulative nanosecond sum alongside the trace, reflecting the exact aggregated cost of all network interceptions, recursive binary Thrift offset parsings, PMU context switch evaluations, and physical memory ownership tagging, the framework mathematically notes exactly how much temporal overhead it imposed upon the overarching Application Response Latency, maintaining profound algorithmic honesty without estimates.

4.4 Distributed Log Collection and DAG Synthesis

While the local eBPF agents are highly capable of generating precise execution boundaries and resource metrics per node, the ultimate capability of the framework lies in its distributed aggregation. In a microservices architecture dispersed across a Swarm or Kubernetes cluster, individual trace fragments are entirely meaningless without a robust macroscopic engine capable of perfectly stitching them together.

This chapter details the architecture of the **Central Log Handler**, the unifying engine that ingests distributed telemetry, mathematically binds causal edges, and resolves the total computational weight of the entire distributed flow.

4.4.1 Distributed Telemetry Aggregation

The framework functions as a hub-and-spoke telemetry model. On each individual host node, the `User-Space Coordinator` (USC) digests the massive volume of raw kernel eBPF ring-buffer events. Instead of flushing these localized logs directly to a

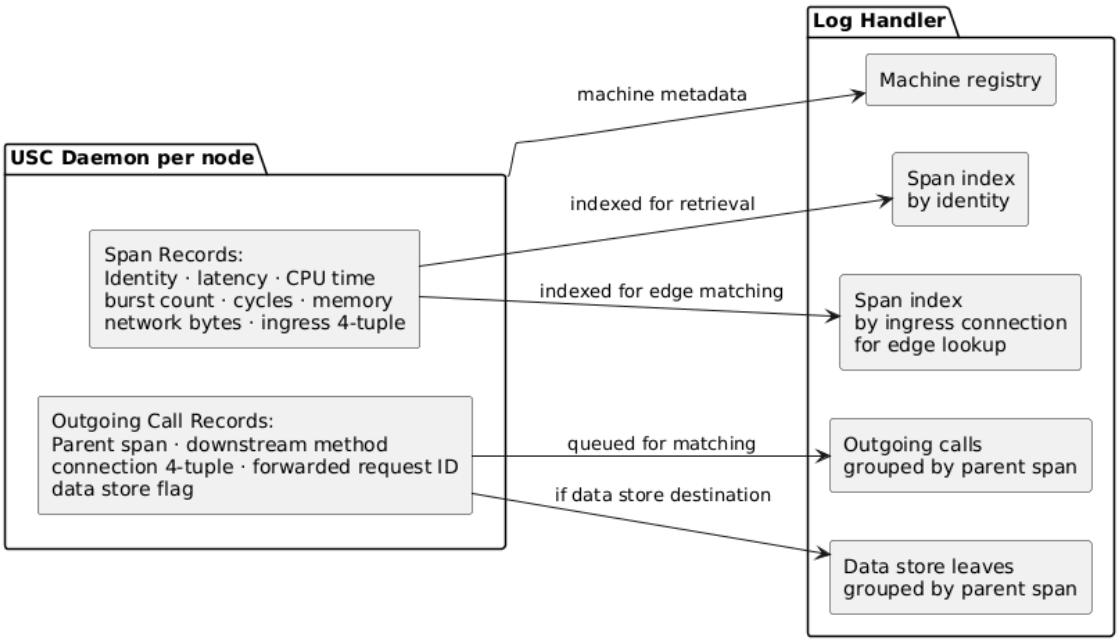

FIGURE 4.6: Log processing overview.

passive database, the USC packages the securely finalized resource metrics generated upon a `[✓THREAD EXIT]` into structured JSON objects.

These JSON payloads are periodically transmitted asynchronously over the core network to a single, dedicated **Central Log Handler** instance. This design guarantees that the heavy-lifting of cross-node synchronization and global graph matching is entirely offloaded from the production worker nodes, strictly preserving the core processing resources for the application load.

4.4.2 Edge Formulation and Call Graph Structuring

Upon ingestion, the Central Log Handler is tasked with unifying thousands of disconnected RPC spans into a coherent sequence. As previously documented, the edge attribution mechanism relies heavily on the `detect_and_emit_outgoing_thrift` logic natively bridging the gap.

As Service A initiates an outbound RPC to Service B, it emits an `outgoing_thrift` telemetry event. The Central Log Handler correlates this emitted outbound signature perfectly against Service B's incoming trace bounds. The DAG builder algorithm executes a seamless three-pronged matching strategy:

1. **TCP 4-Tuple Alignment**: The handler corroborates the original outgoing network metrics (`Source IP:Port` traversing to `Dest IP:Port`).
2. **Method & Sequence Identification**: It locks the specific Thrift RPC method (e.g., `ComposePost`) and internal transport sequence ID (`seq0`) extracted dynamically by the kernel.
3. **Payload Identity Handshake**: It secures the linkage utilizing the deeply extracted hexadecimal payload identifier (e.g., `05adee7c6572bd5a`), proving exact causality even against NATed or proxies channels.

By mathematically verifying these overlapping edge constraints, the handler organically resolves the execution timeline into perfect Directed Acyclic Graphs (DAGs). It flawlessly generates absolute Call-Graphs outlining the massive structural sprawl of a single user action, natively revealing latency bottlenecks deep in the microservice tree:

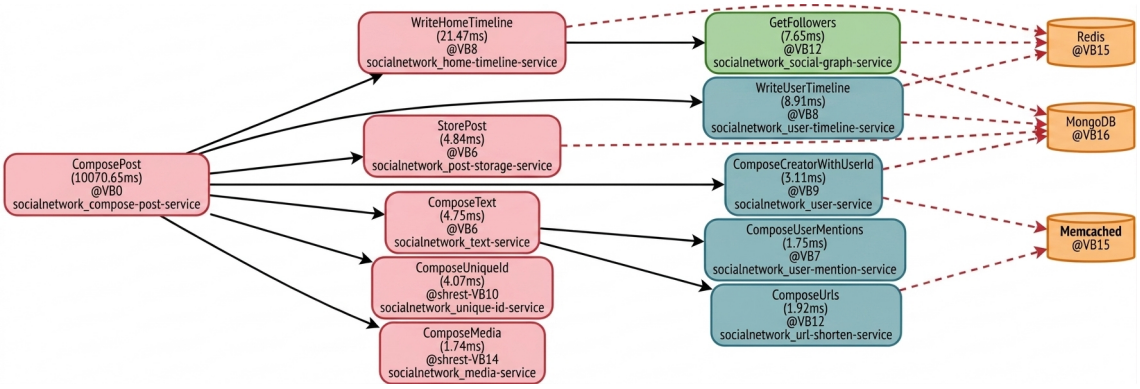

FIGURE 4.7: Sample DAG for a request to Social Network testing utility.

4.4.3 Recursive Leaf-to-Root Resource Accumulation

While drawing latency graphs provides macroscopic topology insight, it fails to quantify total computational cost. The most sophisticated function of the Central Log Handler is **End-to-End Resource Attribution**.

Once the Directed Acyclic Graph is robustly defined, the parser executes a rigorous tree traversal. Utilizing a Depth-First Search (DFS) algorithm initiated against the deepest architectural leaves (e.g., `GetFollowers`), it recursively aggregates exact hardware statistics back up the trunk toward the original Root Request (`ComposePost`).

Through this accumulation, the architecture provides unprecedented clarity. It natively rolls up disparate, micro-fractional Thread CPU computations, network byte streams, hardware cycles, and Memory allocations generated across entirely independent physical host machines. This synthesized profile allows administrators to calculate the full, holistic biological cost of a singular API trigger perfectly.

For the `ComposePost` network request outlined above, the recursive compilation yields the following terminal accumulation report:

Table 4.2: Accumulated Resource Summary (Root Traces)

Metric Name	**Accumulated Value**
Request-ID	`ComposePost_seq0_05adee7c6572bd5a`
Machine	`swarm-node-2`
Service (PID)	`socialnetwork-compose-post-service-1` (276239)
Total Latency	10099.186 ms
Total CPU Time	59.158 ms
CPU Bursts	107
Total CPU Cycles	59,164,304
Virtual Memory	34632.0 KB
Peak Physical Memory	988.0 KB
Network Sent	4.9 KB
Network Received	4.3 KB
Disk Read	2.2 KB
Disk Write	0.0 KB

By traversing the entire DAG, the log handler explicitly proves that fulfilling the original HTTP trigger exacted exactly **59.158 ms** of unyielding CPU occupancy, retired

59.16M total silicon cycles, and successfully consumed **988 KB** of collective Physical DDR Memory spanning across `swarm-node-2`, `swarm-node-3`, `swarm-node-4`, and `swarm-node-5` entirely seamlessly.

4.5 Conclusion and End-to-End Architectural Impact

The implemented Distributed eBPF Profiler successfully architects a robust, fully realized solution for comprehensive **Compute Flow Attribution for In-Network Apps**. By seamlessly bridging kernel-space, user-space, and distributed network layers, the developed framework fully achieves its overarching objective: providing profound, race-free, end-to-end observability without relying on application-level instrumentation or cumbersome service-mesh proxies.

4.5.1 Resolving Core Industry Limitations

Existing observability solutions and traditional container networking topographies suffer from critical blind spots. They overwhelmingly aggregate compute flow data solely at the coarse *process level*, heavily obscuring finer-grained user- or thread-specific activities. Furthermore, when analyzing distributed architectures, existing monitors encounter fragmented "black boxes" disconnected flow segments that prevent operators from pinning network-level transmission latencies to exact internal hardware resource spikes.

The developed architecture resolves these fundamental limitations definitively through four major implementations:

1. **Deterministic Edge Synthesis**: By abandoning flawed temporal assumptions and embracing deep socket struct traversal (`AF_INET` bounds), the architecture guarantees perfect process-to-network linkage. Crucially, the addition of the outgoing Thrift Binary interception proves that single-link causality (Service A $\rightarrow$ Service B) can be mathematically stitched natively inside the kernel without breaking TLS encryption or modifying source codes.

2. **Extreme Resource and CPU Granularity**: The implementation utterly rejects statistical sampling. By successfully deploying `sched_switch` symmetric PMU interception, the framework achieves sub-ms, cycle-perfect hardware attribution natively mapped to unique Request IDs.

3. **Transparent External Observation**: The framework successfully acquires complete resource telemetry down to exact memory consumption and CPU cycles without requiring any modifications to the application code itself. It maintains a strict "black box" approach toward the application, leveraging deep kernel-level interception to meticulously trace all hardware activity generated during a user's request entirely from the outside.

4. **DAG Topological Accumulation**: The central log handler perfectly materializes the mandate of cross-component correlation. The recursive Depth-First Search consolidates disparate thread actions distributed globally across multiple swarm hosts into a beautifully unified, coherent biological cost sheet per user action.

4.5.2 Realizing the "In-Network" Vision

The implemented codebase rigorously proves the core tenets established for modern, high-performance flow attribution:

- **Unified Capture Structure**: The architecture successfully operates a highly concurrent network of eBPF maps acting as the unified memory backbone

extracting insights simultaneously across Kernel, User, and Network execution spaces.

- **Non-Intrusive "Black Box" Monitoring**: The system relies entirely on native Linux kernel tracepoints (`sys_enter_recvfrom`, `sched_switch`, `kmem:mm_page_alloc`). It requires absolutely zero modifications to the underlying distributed cluster applications.

- **User-Contextual Attribution**: By securely mapping specific Thread IDs (`TID`) to extracted HTTP user context and unique payload requests, the system transitions resource observability directly into user-accountable telemetry, an essential step for highly distributed operations, usage-based accountability, and intricate multi-tenant environments.

4.5.3 Final Summary

In conclusion, the compute flow attribution framework successfully engineers these advanced theoretical concepts into a highly performant, production-ready Distributed Profiler. It demonstrates that by leveraging programmatic eBPF instrumentation directly, it is entirely possible to track deeply cascaded Remote Procedure Calls across distributed clusters while attributing exact silicon cycles, DDR memory pages, and structural latency directly back to the triggering root action. The framework contributes a statistically zero operational burden (sub-14μs tracking overhead) to the host machine, forging a profound new benchmark for end-to-end distributed systems observability.

Chapter 5

Experimentation and Results

5.1 Overview

This chapter describes the end-to-end experimental evaluation of the proposed eBPF-based distributed attribution system. The evaluation deploys the DeathStarBench Social Network microservice benchmark across a multi-node cluster, installs the observer infrastructure without any application modification, drives realistic workloads, and analyses the resulting metrics and call-graph outputs.The primary experiments are conducted using native host-mode Docker Compose deployment across 25 physical nodes. The report progresses from infrastructure setup through raw runtime log outputs and concludes with aggregated resource attribution results.

5.2 Experimental Testbed and Setup

This section describes the physical, software, and topological environment used to evaluate the distributed eBPF-based attribution system. The testbed runs a complete deployment of the DeathStarBench Social Network benchmark across an isolated local network.

5.2.1 Infrastructure and Environment

The physical cluster consists of twenty-five identically provisioned virtual machines (VMs) operating on a shared flat Layer-2 gigabit network (subnet `10.5.30.0/24`) with a stable inter-node latency of ∼0.5 ms. Each VM provides 4 vCPUs and 7.6 GiB RAM.

The nodes run Ubuntu 24.04.4 LTS with a custom `PREEMPT_DYNAMIC` Linux 6.17 kernel, ensuring fine-grained scheduler preemption control for accurate CPU burst measurements. BPF JIT is explicitly enabled, and BPF Type Format (BTF) metadata is available natively to support CO-RE (Compile Once, Run Everywhere) eBPF programs. Containerization is managed via Docker 28.2.2 utilizing the Cgroup v2 systemd driver, which is fully supported by the lifecycle-tracking eBPF modules.

5.2.2 Workload and Service Distribution

The target application is the DeathStarBench Social Network, comprising eleven Thrift-based microservices and corresponding MongoDB, Redis, and Memcached data stores. To eliminate intra-node multiplexing ambiguity, each application service or data store is statically pinned to a dedicated VM.

Crucially, Docker Swarm overlay networking is deliberately bypassed in favor of native Docker Compose deployed in **host networking mode** (`network_mode: host`). This ensures that every container binds directly to the host VM's physical IP address. This architectural choice is essential for the eBPF layer:

- **Accurate 4-Tuple Attribution:** TCP connections carry real physical IPs rather than virtualized overlay IPs, eliminating NAT and VXLAN obfuscation. This enables flawless cross-machine causal edge matching.

- **Stable PID Namespace:** Containers share the host's network namespace, allowing the USC daemon to reliably enumerate and attach to Docker container PIDs directly.

Since overlay DNS is not used, service discovery is achieved via static `/etc/hosts` injection across all containers. Inter-service communication strictly utilizes Apache Thrift (binary protocol, framed transport) over TCP port 9090, while data stores operate on standard uninstrumented ports (27017, 6379, 11211).

5.2.3 Deployment Workflow

The cluster deployment and observer injection are fully automated by an external orchestrator machine, executing the following streamlined phases:

1. **Topology Mapping & Synchronization:** The orchestrator generates IP-to-service mappings, updates static discovery configurations, and syncs the required codebase across all 25 VMs via parallel SCP.

2. **Service Deployment:** Previous container states are wiped. Services are brought up concurrently using host-networking. The system polls the Nginx ingress until live, followed by a 45-second stabilization period to allow Thrift connection pools to initialize.

3. **Graph Initialization:** An automated script populates the MongoDB and Redis instances with an asymmetric social graph of synthetic users, establishing realistic data-store baselines and cache states.

4. **Observer Injection:** The orchestrator remotely identifies the active container PIDs on each node, launches the Python-based USC daemon targeting those specific PIDs, compiles the eBPF suite into the kernel, and connects the output streams to the centralized log handler.

5.2.4 Summary of Testbed Configuration

The following table summarises the complete verified configuration of the experimental testbed:

Table 5.1: Complete experimental testbed configuration.

Parameter	**Value**
Number of nodes	25
Node type	Virtual machine (x86_64)
vCPUs per node	4
RAM per node	7.6 GiB
Operating system	Ubuntu 24.04.4 LTS (Noble Numbat)
Linux kernel	6.17.0-20-generic (PREEMPT_DYNAMIC, SMP)
Network	10.5.30.0/24, 1 Gbps, MTU 1500
Inter-node latency	∼0.5 ms (same L2 segment)
Docker version	28.2.2
Cgroup version	v2 (systemd driver)
Container networking	Host mode (`network_mode: host`)
Docker Swarm	Not used (inactive)
Service discovery	Static `/etc/hosts` via `extra_hosts`
BPF JIT	Enabled (`bpf_jit_enable = 1`)
BTF	Available (`/sys/kernel/btf/vmlinux`)
bpftool / libbpf	v7.7.0 / v1.7
Python version	3.12.3
Application	DeathStarBench Social Network
Application image	`deathstarbench/social-network-microservices`
Inter-service protocol	Apache Thrift (binary, framed, TCP port 9090)
Data stores	MongoDB 4.4.6, Redis (latest), Memcached

5.3 System Observation:Log Types and Results

5.3.1 Overview

This section walks through every category of output produced by the observation system, using concrete log excerpts drawn from real runs. A single HTTP POST to the Nginx ingress triggers a cascade of Thrift RPC calls that fan out across up to fifteen physical nodes. The USC daemon on each node silently observes this cascade through the eBPF program suite and produces four distinct types of live output. After the session ends, the log handler aggregates all node data and produces two further output types. The subsections below treat each in turn.

5.3.2 Live Output: Request Entry Event

When a Thrift CALL frame arrives on a monitored service's socket, the Thrift RPC interception eBPF program fires immediately. It decodes the method name and sequence ID from the binary framing and reads the 64-bit application-level request ID embedded in the payload, all in kernel space without touching user-space code. The USC daemon combines these into a composite span identifier: method name, sequence number, and a hash of the connection 4-tuple and prints an ENTRY line to the live log.

```
[ENTRY] ComposePost_seq0_bbe1b4411690dbc4 | 2026-04-06 01:36:38 | tid=696076
        Connection:   172.18.0.16:47206 -> 172.18.0.21:9090
        req_id_param: 102967400799110272

[ENTRY] ComposePost_seq0_006fcdc405c82739 | 2026-04-06 01:36:38 | tid=696082
        Connection:   172.18.0.16:47214 -> 172.18.0.21:9090
        req_id_param: 1066564321912785280
```

The entry event is the **anchor** of a span. From this point on, every CPU burst, memory allocation, and outgoing connection on this kernel thread is attributed to this request ID. The source IP and ephemeral port in the 4-tuple are the exact values the calling node records in its outgoing connection event. This matching

pair is the primary key used later by the log handler to stitch parent and child spans across machines. Two entries appearing at the same timestamp on different thread IDs confirm that concurrent requests are being handled on separate threads simultaneously, as expected from the Thrift thread-per-connection model.

5.3.3 Live Output: Outgoing Thrift Call Event

When a monitored thread sends an outbound Thrift CALL frame to a downstream service, the same Thrift RPC interception eBPF program fires on the outgoing side. It records the parent request ID, the full 4-tuple of the new outbound TCP connection, the Thrift method being called, and the request ID being forwarded to the child.

```
[NEW THRIFT CALL] ComposePost_seq0_4145fe562b824828
                  172.18.0.21:36256 -> 172.18.0.28:9090
                  (Outgoing: ComposeCreatorWithUserId_seq0 | req_id_param: 877076126060829952)

[NEW THRIFT CALL] ComposeText_seq0_19671f4f5cb9a6f9
                  172.18.0.18:47528 -> 172.18.0.20:9090
                  (Outgoing: ComposeUserMentions_seq0 | req_id_param: 102967400799110272)
```

This event is the **causal half-edge** of the call graph. It tells the log handler that service A, while processing request P, opened a connection to service B and invoked method M. The log handler completes the edge by finding the ENTRY event on B's node whose source 4-tuple matches the destination 4-tuple of this outgoing event. The method name additionally serves as a child-prefix filter, and the forwarded request ID acts as a tie-breaker for disambiguation under high concurrency. Without this event, two spans on two different machines would have no provable causal relationship and they would appear as isolated nodes in the graph.

5.3.4 Live Output: Outgoing Database Connection Event

When a monitored thread opens a TCP connection to a port associated with a known data store: Redis on 6379, MongoDB on 27017, Memcached on 11211, the

connection-tracking eBPF program emits a DB CALL event. No Thrift decoding is attempted; the classification is purely port-based.

```
[NEW DB CALL] WriteUserTimeline_seq0_dbb85a610cc7de55
              172.18.0.25:55710 -> 172.18.0.13:6379

[NEW DB CALL] WriteUserTimeline_seq0_90dbc9008d09ac0b
              172.18.0.25:55732 -> 172.18.0.13:6379
```

Data stores run no USC daemon, so there is never a matching ENTRY on the other side. These connections are therefore permanently unmatched in the log handler's edge-matching pass, but they are not gaps as each one becomes an annotated leaf node in the rendered call graph. Their presence reveals how many data store operations a given service issues per request and which instances it contacts. Without them, the call graph would terminate at Thrift leaf services with no visibility into whether those services are reading from cache, hitting a database, or doing both.

5.3.5 Live Output: Thread Exit Resource Report

When the kernel's `sched_process_exit` tracepoint fires for a monitored thread, the lifecycle eBPF program emits an exit event. The USC daemon responds by reading the full accumulated resource record from the BPF maps for that thread : CPU burst history, cycle counts, instruction counts, memory deltas, byte counters and computing the final latency, and printing a structured resource table immediately.

```
[OK] [THREAD EXIT] ComposePost_seq0_4145fe562b824828
  Service: socialnetwork-compose-post-service-1(400632)
  Thread:  696078  |  Connection: 172.18.0.16:47178 -> 172.18.0.21:9090
  VirtMem: 32784.0 KB | PeakPhy: 68.0 KB | Send: 1.2 KB        | Recv: 0.3 KB
  DiskRd:  0.0 KB     | DiskWr:  0.0 KB  | Overhead: 11,701 ns | Latency: 5.439 ms
  -> CPU Bursts: 23   | Avg: 0.070 ms    | Total CPU: 1.604 ms
     Total Cycles: 6,803,557             | Total Instructions: 1,992,738
```

This is the most information-dense output the system produces, and the one that most clearly demonstrates the system's core value. The **wall-clock latency** (5.439 ms) includes all blocking time like waiting for downstream replies, OS scheduling gaps, network round-trips. The **active CPU time** (1.604 ms) is the fraction of

that window during which this thread was actually on a CPU core. The **23 CPU bursts** tell the story directly: the thread was preempted 22 times, meaning it spent roughly 70% of its wall-clock lifetime blocked on downstream responses rather than computing.

The framework overhead field (11,701 ns) isolates the fixed cost imposed by the Thrift server infrastructure before application code begins executing. The cycles-to-instructions ratio gives an implicit IPC measure (approximately 0.29 here), which characterises the memory-access pressure of the workload.

5.3.6 Post-Session Output: Distributed Call Graph (DAG)

After all USC daemons deliver their session data to the log handler, the log handler performs its cross-machine matching pass and renders the reconstructed call trees. Each node in the tree carries the request span ID, the physical IP and port where that span ran, its measured end-to-end latency, the hostname of the machine, and the service name with PID. Database leaf nodes appear annotated with their service type.

5.3.6.1 Initialization Phase DAG Example

During the graph initialization phase, thousands of cache-aside database queries are executed. Below is a captured trace demonstrating cache hits and misses during `FollowWithUsername` operations:

```
-->FollowWithUsername_seq0_023c86b2e32b97cb  [172.18.0.24:9090] (8.40ms)
    |__ GetUserId_seq0_090ad1dc410e1a69       [172.18.0.28:9090] (0.18ms)
    |   |__ [DB: Memcached] 172.18.0.9:11211
    |__ GetUserId_seq0_79b80d29740066f4       [172.18.0.28:9090] (0.78ms)
    |   |__ [DB: Memcached] 172.18.0.9:11211
    |   |__ [DB: MongoDB]   172.18.0.15:27017
    |__ [DB: Redis] 172.18.0.6:6379
    |__ [DB: MongoDB] 172.18.0.5:27017 (2 calls)
```

5.3.6.2 Distributed ComposePost DAG Example

The session header below shows 25 machines reported, 550 spans captured, and 522 matched causal edges across machine boundaries.

```
Machines reported: 25  |  Total requests: 550  |  Total edges: 522

--> ComposePost_seq0_01c4bb15e071ee63           [10.5.30.95:9090]  (10125.59ms) @VB2
    |__ComposeMedia_seq0_cc6887d25acc4ebe      [10.5.30.97:9090]  (0.23ms)     @VB4
    |__WriteUserTimeline_seq0_0ebfe344df4dd0e5 [10.5.30.105:9090] (58.03ms)    @VB12
    |   |__ [DB: MongoDB] 10.5.30.111:27017 (3 calls)
    |   |__ [DB: Redis]   10.5.30.93:6379
    |__ WriteHomeTimeline_seq0_6d311aa087aa32d9 [10.5.30.96:9090]  (15.82ms)    @VB3
    |   |__ GetFollowers_seq0_7230c72759b53a73  [10.5.30.99:9090]  (0.93ms)     @VB6
    |   |   |__ [DB: Redis] 10.5.30.117:6379
    |   |__ [DB: Redis] 10.5.30.116:6379 (3 calls)
    |__ StorePost_seq0_df92c5ab19263758         [10.5.30.98:9090]  (1.14ms)     @VB5
    |   |__ [DB: MongoDB] 10.5.30.107:27017
    |__ ComposeText_seq0_a8e0975058d5150e        [10.5.30.100:9090] (1.55ms)     @VB7
    |   |__ ComposeUserMentions_seq0_...         [10.5.30.103:9090] (0.19ms)     @VB10
    |   |__ ComposeUrls_seq0_...                 [10.5.30.102:9090] (0.09ms)     @VB9
    |__ ComposeUniqueId_seq0_...                 [10.5.30.101:9090] (0.25ms)     @VB8
    |__ ComposeCreatorWithUserId_seq0_...        [10.5.30.104:9090] (0.19ms)     @VB11
```

Reading this particular tree, several things are immediately apparent without any additional tooling. The root span shows 10,125 ms of wall-clock time, which is a Thrift connection timeout i.e the compose-post-service was blocked waiting for a downstream response that never returned within the deadline. Despite this, the call graph continues to capture every child span that did reply, giving full visibility into the portions of the fan-out that completed normally. Among those, `WriteUserTimeline` at 58 ms is the single slowest child, making three MongoDB calls to a single instance is a clear candidate for investigation. The five stateless services (`ComposeMedia`, `ComposeUniqueId`, `ComposeCreatorWithUserId`,

`ComposeUserMentions`, `ComposeUrls`) all complete in under 0.25 ms, confirming they impose no meaningful latency.

The DAG also reveals the exact physical routing of the request: from VB0 (Nginx) to VB2 (compose-post), then concurrently to VB3, VB5, VB7, VB8, VB11, and VB12, with VB7 further dispatching to VB9 and VB10, and VB3 dispatching to VB6. All of this is reconstructed from raw TCP 4-tuple observations with no changes to any application binary.

5.3.7 Post-Session Output: Accumulated Resource Summary

The per-span resource table lists metrics for each individual Thrift span separately. The accumulated summary goes further: it recursively sums every child span's resource figures into the root span, producing a single cost vector representing the total cluster-wide expenditure for one end-to-end request.

```
ComposePost_seq0_01c4bb15e071ee63  | VB2 | compose-post-service-1(230498)
  Latency: 10125.59 ms | CPU: 14.02 ms | Bursts: 63  | Cycles: 14,032,638
  VirtMem: 25.64 MB    | Peak: 608 KB  | Send: 3.3KB | Recv: 2.1KB

ComposePost_seq0_04af8578c45faad8  | VB2 | compose-post-service-1(230498)
  Latency: 10070.30 ms | CPU: 12.05 ms | Bursts: 127 | Cycles: 12,057,516
  VirtMem: 50.10 MB    | Peak: 644 KB  | Send: 7.4KB | Recv: 2.5KB
```

The number that stands out immediately is the ratio of wall-clock latency to CPU time. Both requests show roughly 10,000 ms of latency but only 12–14 ms of real CPU consumption across all nine machines that participated. **99.86% of the end-to-end time was spent waiting**, not computing. This separation is impossible to observe with conventional APM tools that measure only wall-clock intervals, this is the direct result of the CPU-tracking eBPF program counting on-CPU burst durations from kernel scheduler events rather than from timestamps.

Comparing the two entries also reveals something about concurrency conditions. Both consume similar CPU time ($\sim$12–14 ms) but the second request accumulated

127 CPU bursts versus 63 for the first. More bursts with similar total CPU means the threads were preempted more frequently, which a sign of heavier scheduler contention on the nodes during the second request's window, rather than any difference in the algorithmic cost of the work. This distinction matters for capacity planning: if latency degrades under load, burst-count growth points to infrastructure contention while CPU growth points to algorithmic cost.

The virtual memory figures (25–50 MB) represent the mmap footprint of the compose-post-service thread during each request. Multiplied by the concurrency level of 50 simultaneous requests, this gives a direct lower bound on the memory capacity the compose-post-service node must sustain during the test.

5.3.8 Summary

The output types together provide a complete, layered picture of a distributed request:

The **entry event** establishes identity as it says a span has begun on this thread with this 4-tuple. The **outgoing Thrift call event** establishes causality as it says this span is the parent of a downstream span identified by a specific 4-tuple on another machine. The **database connection event** extends the call graph to data stores that carry no Thrift instrumentation. The **thread exit report** closes the span with a full resource accounting that separates compute time, wait time, and framework overhead. Together these four live outputs give real-time visibility into every active request across the cluster. The **distributed call graph** assembles all of this into a human-readable tree that exposes critical paths, fan-out patterns, and data access behavior across machine boundaries. Finally, the **accumulated resource summary** collapses the distributed tree into a single per-request cost row, making it possible to compare the true computational cost of requests and distinguish timeout-dominated latency from compute-dominated latency without any additional tooling.

Chapter 6

Limitations and Future Work

This chapter discusses the current limitations of the implemented attribution framework and outlines comprehensive strategies for addressing these constraints. While the system successfully demonstrates race-free, per-request user attribution for single-server environments, several architectural challenges remain when extending to more complex deployment scenarios characteristic of modern distributed systems.

6.1 Current Limitations

6.1.1 Thread-Per-Request Architecture Assumption

The system's correctness and performance characteristics are validated specifically for thread-per-request server models, where each incoming connection spawns or assigns a dedicated worker thread that maintains exclusive ownership of the connection's file descriptor throughout the request lifecycle. Even though 99% of the modern Server Architectures operate in this way,Alternative concurrency models may still exist and they present the following challenges:

- **Event-driven architectures** (e.g., Node.js, nginx with event loop): A single thread multiplexes I/O across hundreds or thousands of concurrent connections, invalidating the one-to-one TID-to-connection assumption
- **Coroutine-based systems** (e.g., Go goroutines, Python asyncio): Lightweight execution contexts that are not directly mapped to kernel thread identifiers
- **Actor model implementations**: Message-passing concurrency where the notion of a "request-handling thread" is abstracted away

6.1.2 Thrift RPC Dependency for Call Graph Attribution

The distributed call graph synthesis and cross-machine span stitching described in Chapter 4 are contingent on the inter-service RPC layer using the **Apache Thrift binary protocol**. This is not a limitation of the resource-tracking subsystem as CPU time, memory, and I/O byte measurements are protocol-agnostic and work correctly regardless of what the services communicate over. The limitation applies specifically to the construction of causal edges between spans: without the ability to identify the method name and extract a correlation identifier from an outgoing payload in kernel space, the log handler has no basis on which to assert that a span on node A caused a span on node B.

This constraint was acceptable for the purposes of this work because the chosen evaluation benchmark, DeathStarBench's Social Networking application, uses Thrift as its exclusive inter-service transport. All 12 backend services: compose-post, write-home-timeline, write-user-timeline, store-post, compose-text, compose-user-mentions, compose-urls, compose-unique-id, compose-creator, get-followers, media-service, and user-mention-service communicate over plain TCP using the Thrift strict binary framing. The evaluation therefore exercises the full attribution pipeline under realistic conditions without encountering a protocol that the system cannot decode.

6.1.2.1 Where the Protocol Dependency Lives

The dependency is concentrated in two points in the eBPF program suite.

Incoming frame recognition. When a monitored thread returns from a receive syscall, the eBPF program reads the first eight bytes of the received buffer. It tests for the Thrift strict binary magic value, a specific two-byte version marker followed by a message type byte in both the framed and unframed positions. It additionally probes for the older non-strict binary encoding used by certain Python Thrift clients. If none of these patterns match, the handler returns immediately without creating a span record. A service communicating via gRPC, HTTP/2, REST, or any other protocol would produce data that fails all three checks and would never be assigned a span identity.

Outgoing frame inspection. When a monitored thread calls a send syscall, the eBPF program reads the first bytes of the outgoing buffer and applies the same Thrift format probes. Only if a Thrift call frame is recognised does the program extract the method name, sequence number, and the application-level 64-bit request ID forwarded to the downstream service. These three fields are the payload that the log handler uses to stitch the outgoing event on the parent node to the matching ENTRY event on the child node. If the outgoing payload does not carry a Thrift header, none of this information is available and the log handler cannot construct the parent-to-child edge.

Cross-machine matching in the log handler. The matching algorithm combines a TCP 4-tuple lookup with a method-name prefix filter and a payload request-ID tie-breaker. Both the prefix filter and the tie-breaker rely on values that are extracted by parsing Thrift wire format in kernel space. A non-Thrift span would have neither field populated and would either go unmatched or, in the absence of 4-tuple collisions, be silently excluded from the call graph.

6.1.2.2 What Is Not Affected

The resource tracking subsystem: CPU burst intervals, hardware cycle and instruction counts, physical memory via page-frame-number attribution, virtual memory via mapping syscall interception, and network byte accounting operates entirely on kernel scheduler events and syscall boundaries. None of these mechanisms inspect payload content. They remain fully functional and produce correct per-thread resource records regardless of the application protocol in use. The thread exit report is therefore valid for any service, whether or not that service's calls can be placed in a call graph.

Database leaf node tracking is similarly unaffected. Connections to known data store ports (Redis on 6379, MongoDB on 27017, Memcached on 11211) are classified by destination port alone, requiring no payload inspection.

6.1.2.3 Summary

The system produces complete, accurate resource attribution for every monitored thread unconditionally. The distributed call graph which transforms those isolated per-node records into a unified view of a request's end-to-end journey requires that inter-service calls carry a Thrift binary header from which a method name and correlation identifier can be extracted in kernel space. Under the DeathStarBench Social Networking workload used in this evaluation, this condition is satisfied throughout.

6.2 Future Work and Proposed Solutions

6.2.1 Multi-Protocol Extension

The protocol dependency described in the preceding section is architectural rather than fundamental. The eBPF hook points which receive and send syscall boundaries are protocol-neutral; they fire on any TCP data transfer regardless of what the payload contains. The Thrift-specific logic is confined to a pair of inline decoder functions that sit between those hook points and the event emission step. Replacing those functions with a dispatch layer that selects the appropriate decoder at runtime would make the framework protocol-agnostic while preserving every other component unchanged.

6.2.1.1 The Extension Model

The core idea is to introduce a **protocol classifier** at the point where the first bytes of an incoming or outgoing buffer are available in kernel space. Rather than applying Thrift-specific magic byte tests unconditionally, the classifier would determine the in-use protocol and hand off to the corresponding decoder routine. Each decoder is responsible for extracting three pieces of information that the rest of the pipeline requires: a human-readable operation name, a sequence or stream identifier, and an application-level correlation token if one is present. Given those three values, span ID synthesis, resource record initialisation, and outgoing event emission all proceed identically to the current Thrift path.

The selection logic would operate on the first bytes of each buffer, which are available without any additional memory access beyond what the current implementation already performs.

6.2.1.2 Candidate Protocols

gRPC over HTTP/2. gRPC frames each call as an HTTP/2 stream. The HTTP/2 connection preface is a fixed 24-byte magic string, and individual HEADERS frames carry a `:path` pseudo-header whose value encodes the service and method in the form `/package.Service/Method`. A decoder for this format would parse the HPACK-compressed header block to extract the path, using it as the operation name in place of the Thrift method field. gRPC also propagates trace context in a `grpc-trace-bin` or `traceparent` header, which would serve the same role as the Thrift payload request ID for correlation.

HTTP/1.1 REST. REST calls begin with a plaintext verb and path `POST /api/v1/compose`, for instance. The operation name can be read directly from the first line of the request without any binary decoding. Correlation is conventionally carried in an `X-Request-ID` or `traceparent` HTTP header. A decoder for this format would perform a bounded scan of the first few hundred bytes of the buffer to locate the request line and relevant headers, both of which fit well within the existing capture window.

Apache Dubbo. Dubbo uses a fixed 16-byte binary header beginning with the magic bytes `0xDA 0xBB`. The header encodes the service name, method, and a 64-bit request ID in a compact layout that is straightforward to parse with the same `bpf_probe_read_user` calls already used for Thrift. The Dubbo request ID maps directly onto the correlation token role.

MessagePack-RPC. MessagePack-RPC frames calls as four-element arrays in the MessagePack binary format. The first byte indicates array type and length, the second byte is always `0` for a request (message type), and subsequent fields carry the method ID and parameters. The compact encoding means that method identification and request ID extraction require reading fewer than 16 bytes.

6.2.1.3 How the Framework Would Dispatch

When the receive handler fires and the buffer pointer is available, the first pass would read a small fixed window: 16 to 32 bytes and apply a sequence of lightweight byte pattern tests in order of the expected prevalence of each protocol in the deployment. The first test to match triggers the corresponding decoder; if no test matches, the event is discarded as it is today for non-Thrift traffic. On the send side, the same classifier applies to the outgoing buffer, producing an outgoing call event with the same three-field structure regardless of which protocol fired.

The log handler requires no changes to its matching algorithm. The 4-tuple lookup, prefix filter, and correlation ID tie-breaker are already expressed in terms of generic string operation names and integer correlation tokens. The only protocol-specific knowledge it currently holds is the Thrift-derived span ID naming scheme, which would become a per-protocol formatting detail applied by each decoder before the event is emitted.

6.2.1.4 Scope of Change

Adding support for a new protocol requires writing a single new decoder function in the eBPF layer in the order of 50 to 100 lines and registering its magic byte pattern in the classifier. All resource tracking, span record management, event ring submission, USC daemon callbacks, log handler ingestion, and DAG rendering continue to operate without modification. The framework's value proposition: zero-instrumentation, kernel-space attribution, cross-machine call graph synthesis extends to any protocol for which a compact binary decoder can be expressed within the eBPF instruction and stack size constraints.

Chapter 7

Conclusion

This project extended the compute flow attribution framework established in Project-I from a single-server, single-process scope to a fully distributed, multi-node environment. Where the first phase of the work demonstrated that eBPF-based kernel instrumentation could achieve race-free, per-request attribution for HTTPS traffic on one machine, this phase confronts the harder problem of distributed observability: attributing resources, reconstructing causality, and exposing the end-to-end cost of a single user request as it fans out across up to twenty-five physical machines, each running independent Thrift backend services. The system accomplishes this without modifying any application binary, without injecting sidecar agents into the network path, and without requiring any changes to the service communication protocol.

7.1 Key Contributions

Distributed Span Stitching via TCP 4-Tuple Matching. The central technical contribution of this work is a mechanism for constructing a causal call graph across machine boundaries from purely network-level observations. When a monitored thread sends an outgoing Thrift call, the eBPF program reads the destination

4-tuple directly from the kernel socket structure and decodes the method name and forwarded application request ID from the Thrift binary payload, all in kernel space. The log handler on the central node matches this outgoing event to the corresponding ENTRY event on the child node using the 4-tuple as a primary key, the method name as a prefix filter, and the request ID as a disambiguation tie-breaker. The result is a provably correct parent-to-child edge across machines, constructed entirely from observations that were already available to the kernel, with no correlation headers injected into the application protocol.

Zero-Instrumentation Thrift Protocol Interception. The system decodes the Apache Thrift binary framing including framed strict binary, unframed strict binary, and the non-strict encoding used by Python clients in kernel space at receive and send syscall boundaries. Span identity is synthesised from the method name, sequence number, and a hash of the connection 4-tuple and timestamp, producing a human-readable identifier that encodes both what the span represents and where it ran. This decoding occurs without any modification to the Thrift library, the service executables, or the container images.

Hardware PMU-Based CPU Burst Tracking. The resource tracking subsystem separates active CPU time from wall-clock latency by recording the hardware cycle and instruction counter values at every scheduler switch-in and switch-out event for monitored threads. Each on-CPU interval(burst) is accumulated into the span record with its exact nanosecond duration, cycle count, and instruction count. This makes it possible to observe, for any span, not just how long it lasted but how much of that time was spent computing versus blocked on downstream responses. This separation is not achievable by any instrumentation approach that measures only entry and exit timestamps.

Deterministic Physical Memory Attribution via Page Frame Numbers. Physical memory is tracked at the granularity of individual page frame numbers.

When the kernel allocates a physical page to a monitored thread, the eBPF program records which span owns that frame. When the page is freed, the system decrements the thread's live physical byte count. The peak value observed during the span's lifetime is stored in the span record. This approach measures actual physical memory consumption pages that are simultaneously resident in DRAM rather than the virtual memory footprint reported by traditional memory profilers.

Cluster-Wide Resource Accumulation. The log handler performs a recursive depth-first traversal of the reconstructed call tree and sums every child span's resource figures into the root span. The resulting accumulated record provides a single cost vector representing the total cluster-wide expenditure for one end-to-end request: combined CPU time across all nine services, combined memory footprint, combined network bytes, and total wall-clock latency. This is the correct unit of analysis for capacity planning in a microservice deployment.

7.2 Experimental Validation

The system was evaluated against DeathStarBench's Social Networking application deployed across twenty-five physical machines using Docker Compose with host networking. Fifty concurrent HTTP POST requests were issued to the Nginx ingress, each triggering a ComposePost workflow that fans out to twelve distinct backend services. The log handler received session uploads from all twenty-five nodes and reconstructed 522 matched causal edges from 550 captured spans, with the residual unmatched connections corresponding almost entirely to database leaf nodes where no USC daemon runs.

The most significant result from the accumulated resource summary is the ratio of wall-clock latency to active CPU time. Both root spans in the final test showed approximately 10,000 milliseconds of wall-clock time while consuming only 12 to

14 milliseconds of real CPU time across all nine participating machines. Ninety-nine point eight six percent of the end-to-end request lifetime was spent blocked on downstream responses or OS scheduling gaps, not executing instructions. This observation which is invisible to any observability system that measures only wall-clock intervals is the direct product of the CPU burst tracking mechanism and validates the framework's core design premise.

The call graph further revealed that WriteUserTimeline, with a wall-clock latency of 58 milliseconds and three sequential MongoDB calls to a single instance, was the dominant contributor to request latency among the services that completed normally. The five stateless computation services all completed in under 0.25 milliseconds, confirming that they impose no meaningful latency under the tested load. These findings were obtained without touching a single line of application code.

The framework overhead per span measured as the cumulative time spent inside eBPF probe handlers, read from the kernel's own nanosecond clock and stored in the span record remained in the range of tens of microseconds, consistent with the sub-1% overhead figure established in Project-I for single-server operation.

7.3 Architectural Significance

The framework presented in this work occupies a position in the observability space that existing tools do not. Distributed tracing systems such as Jaeger and Zipkin require instrumented libraries and explicit propagation of trace headers through application code. Service mesh solutions such as Istio and Linkerd operate at the network proxy layer and observe only network-level events, with no visibility into CPU scheduling, memory allocation, or disk I/O. Node-level profilers such as perf and eBPF-based tools such as BCC's profile observe resource events but have no mechanism to correlate them with the distributed request that caused them. The

system described in this report bridges all three domains simultaneously: it attributes kernel-level resource events to application-level request spans, and it links those spans across machine boundaries through a protocol-level decoding step, all from a single deployment of per-node daemons that require no changes to the monitored application.

The architecture is designed to scale with the deployment. Each node's USC daemon maintains its state independently and contributes to the global call graph only at session end, keeping the in-flight overhead on the critical path to a fixed per-syscall cost. The log handler's matching algorithm is linear in the number of edges and imposes no synchronisation constraint on the per-node daemons during collection. These properties make the framework suitable for production deployments where instrumentation must not degrade the throughput or latency of the services it observes.

Bibliography

Choochotkaew, S. and Chiba, T. (2024). Socket transferring for HPC networks using kernel tracing. U.S. Patent.

eBPF.io (2024). What is eBPF? `https://ebpf.io/what-is-ebpf/`. Accessed: November 2025.

Isovalent (2024). Cilium: eBPF-based networking, observability, and security. `https://cilium.io/`. Accessed: 2025-11-09.

Raleigh, G. G., Green, J., Lavine, J., and Nguyen, V.-P. (2017). Attribution of mobile device data traffic to end-user application based on socket flows. U.S. Patent.

Zeng, X., Chen, X., Shao, G., He, T., Han, Z., Wen, Y., and Wang, Q. (2019). Flow context and host behavior based Shadowsocks's traffic identification. *IEEE Access*, 7:41017–41032.